PROCESS AUTOMATION · CALL CENTERS

List Filter

A workflow to clean and prioritize contact lists.

n8n | JavaScript | Node.js | Python | Data Processing

200	86	114	7.6 h
contacts processed	callable out of 200	discarded with a reason	of operator time saved

THE SITUATION

Call centers receive contact lists and dial them top to bottom: duplicates, invalid phone numbers, people outside the coverage area. Before dialing a single number, they've already burned time on contacts that were never going to connect. The deliverable isn't the clean list — it's the number. How many came in, how many were left usable, and how much calling time is saved, with the assumption in plain sight.

WHAT I FOUND

Out of 200 test contacts, **82 had no usable phone number**, 14 were duplicates and 39 fell outside the coverage area. Only **86 were worth calling**.

My CUIL validator reported **200 out of 200 valid**. Checking it against the actual AFIP rule, I found it was calibrated to the file's generator, not to reality. The fix dropped undetected typos **from 1.58% to 0%**.

The regression suite was green, but several checks weren't running the pipeline — they were reading files from earlier runs. I broke a rule on purpose and it stayed green. I fixed it and tested it for real.

WHAT I BUILT

An n8n workflow with **10 chained nodes** that normalizes phone numbers, validates CUILs with mod-11, deduplicates by CUIL and by phone, flags coverage area and record age, assigns an **explainable score per row** (every point leaves its reason) and produces an impact report.

Safety net: **Python verifiers**, comparison against *golden files* by SHA-256, and adversarial files written to break the pipeline that surfaced 13 documented findings. No row is dropped silently: a discard is a label, not a deletion.

View the repository →